



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Lập trình Hướng Đối tượng Object Oriented Programming

\$11.B Lập trình tổng quát
Generic Programming





Nội dung

- Lập trình tổng quát
- Lập trình tổng quát trong C#





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Lập trình tổng quát



Lập trình tổng quát tại sao

Quản lý danh sách số nguyên =>

```
public class IntList {  
    public void Add(int item) {...}  
}  
  
static void Main() {  
    IntList list1 = new IntList();  
    list1.Add(1);  
}
```

Nhu cầu quản lý danh sách chuỗi, đối tượng khác ?

```
public class TextList {  
    public void Add(string item) {...}  
}  
  
public class ObjectList {  
    public void Add(objectc item) {...}  
}
```

Giải pháp: lập trình tổng quát

```
public class GenList<T> {  
    public void Add(T input) {...}  
}
```

```
static void Main() {  
    GenList<int> list1 = new GenList<int>();  
    list1.Add(1);  
}
```

Lập trình tổng quát

Lập trình tổng quát(Generic programming)

- Là phong cách lập trình tổng quát hóa, trong đó thuật toán được cài đặt có thể làm việc với các kiểu dữ liệu/đối tượng khác nhau, không cần phải chỉ rõ kiểu dữ liệu cho đến khi mã được sử dụng
- Đoạn mã có thể làm việc với các kiểu dữ liệu khác nhau nhưng vẫn đảm bảo kiểm tra kiểu khi biên dịch (compile-time type checking)
- Giảm các đoạn mã trùng lặp, tăng khả năng tái sử dụng mã, kiểu an toàn (type-safety)
- Mã tổng quát và hiệu quả hơn

Lập trình tổng quát

- Lập trình tổng quát(Generic programming): Tổng quát hóa chương trình để có thể hoạt động với các kiểu dữ liệu khác nhau, kể cả kiểu dữ liệu trong tương lai
 - Thuật toán đã xác định

Phương thức **sort()**

- Số nguyên int
- Xâu ký tự String
- Đối tượng sinh viên
- ...



Thuật toán giống nhau, chỉ khác về kiểu dữ liệu

Lớp lưu trữ kiểu ngăn xếp

- Lớp IntegerStack: đối tượng Integer
- Lớp StringStack : đối tượng String
- Lớp AnimalStack: đối tượng animal
- ,...



Các lớp có cấu trúc tương tự, khác nhau về kiểu đối tượng xử lý

Lập trình tổng quát

- Tổng quát hóa chương trình để có thể hoạt động với các kiểu dữ liệu khác nhau, kể cả kiểu dữ liệu trong tương lai
 - Thuật toán đã xác định
- Ví dụ cài đặt
 - C: con trỏ void
 - C++: template
 - C#: template
 - Java 1.5: template

Ví dụ: C dùng con trỏ void

- Hàm memcpy

```
void * memcpy(void * region1, const void* region2, size_t n)
{
    const char* first = (const char*)region2;
    const char* last = ((const char*)region2) + n;
    char* result = (char*)region1;
    while (first != last)
        *result++ = *first++;
    return result;
}
```

Template

- Là mẫu, không phải là một kiểu dữ liệu cụ thể
Như một vị trí dành sẵn (place holder)
viết mã chưa cần xác định thao tác với kiểu dữ liệu nào
- Khi mã sử dụng biên dịch - trình biên dịch copy mẫu, đặt kiểu dữ liệu cụ thể tương ứng vào vị trí
vẫn cho phép compiler kiểm soát tương thích kiểu



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

2. Lập trình tổng quát trong C#

Generic programming in C#

Định nghĩa generic

- Vị trí của generic: trong cặp dấu < >: <tên ảo>
 - Tên: thường bắt đầu chữ cái T, U, V



Các loại generic

- Generic type parameter
- Generic Class
- Generic Interface
- Generic Method
- Generic Delegate



Generic type parameters

- Như một kiểu dữ liệu => khai báo biến - kiểu T là động

```
public int IComparer<T>() { return 0; }
```

Generic classes

- Generic class cho phép chúng ta định nghĩa một lớp (đóng gói xử lý) mà hoạt động với một kiểu dữ liệu chưa biết trước
 - Thường áp dụng cho collection chẳng hạn như List<T>
 - Các phương thức hay thuộc tính của lớp tổng quát có thể sử dụng các kiểu được khai báo như mọi lớp bình thường khác
- Nếu nhiều class có chung logic, chỉ khác biệt về kiểu dữ liệu xl
- Định nghĩa: sử dụng <> class class_name<T>

```
class DataStore<T>
{
    public T data;
}
```

Generic classes

- VD: Danh sách lưu các phần tử cùng là kiểu bất kỳ

```
class List<T>
{
    private T[] _data;
    public int Count => _data.Length;
    public List(int size) => _data = new T[size];
    public void Set(int index, T value) {
        if (index >= 0 && index < _data.Length) _data[index] = value;
    }
    public T Get(int index) {
        if (index >= 0 && index < _data.Length) return _data[index];
        return default(T);
    }
}
```

Generic classes

- VD: Danh sách lưu các phần tử cùng là kiểu bất kỳ

```
static void Main(string[] args)
{
    var listInt = new List<int>(10);
    for (var i = 0; i < listInt.Count; i++)
        Console.WriteLine($"{listInt.Get(i)}t");

    var listChar = new List<char>(5);
    for (var i = 0; i < listChar.Count; i++)
        Console.WriteLine($"{listChar.Get(i)}t");

    Console.ReadKey();
}
```

Generic method

- Phương thức/method có sử dụng kiểu tổng quát (tham số, kiểu trả về)
 - Có thể được viết trong lớp bất kỳ (tổng quát hoặc không)
 - Method có thể có cả generic và non-generic parameter
 - Method có thể nạp chồng

```
public T GetData(int index)
```

```
public void AddOrUpdate(int index, T data) { }
```

```
public void AddOrUpdate(T data1, T data2) { }
```

```
public void AddOrUpdate<U>(T data1, U data2) { }
```

```
public void AddOrUpdate(T data) { }
```

Generic method

- VD

lớp thao tác array

```
class DataStore<T>
{
    private T[] _data = new T[10];
    public void AddOrUpdate(int index, T item) {
        if(index >= 0 && index < 10)
            _data[index] = item;
    }
    public T GetData(int index)
    {
        if(index >= 0 && index < 10)
            return _data[index];
        else
            return default(T);
    }
}
```

Generic method

■ VD

lớp thao tác array

```
public static void Main()
{
    DataStore<string> cities = new DataStore<string>();
    cities.AddOrUpdate(0, "Hanoi");
    cities.AddOrUpdate(1, "HaiPhong");
    cities.AddOrUpdate(2, "DaNang");

    Console.WriteLine(cities.GetData(1));

    DataStore<int> empIds = new DataStore<int>();
    empIds.AddOrUpdate(0, 50);
    empIds.AddOrUpdate(1, 65);
    empIds.AddOrUpdate(2, 89);

    Console.WriteLine(empIds.GetData(0));
}
```

```
> HaiPhong
50
```

Generic Interface

- Generic interfaces cho phép chúng ta định nghĩa một interface mà hoạt động với một kiểu dữ liệu chưa biết trước.

```
public interface IMyGenericInterface<T>
{
    T GetValue();
}
```

Generic Interface

- VD:

cần lưu đối tượng xuống DB

```
class Book
{
    public string Name { get; set; }
    public int Page { get; set; }
}
```

```
class Author
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

```
interface IBaseRepository<T>
{
    List<T> getAll();
    T GetById(int id);
    T Save(T item);
}

class BaseRepository<T>: IBaseRepository<T>
{
    public List<T> getAll(){
        return new List<T>();
    }
    public T GetById(int id){
        //Truy vấn từ DB...
        return item;
    }
    public T Save(T item){
        //Lưu xuống DB...
        return item;
    }
}
```

Generic Interface

- VD: cần lưu đối tượng xuống DB

```
static void Main(){
    BaseRepository<Book> bookRepository = new BaseRepository<Book>();
    BaseRepository<Author> authorRepository = new BaseRepository<Author>();

    Book book = bookRepository.Save(new Book { Name = "Book1", Page = 100 });
    Author author = authorRepository.Save(new Author { Name = "Author1", Age = 50 });

    Console.WriteLine("Book: {0} - page: {1}", book.Name, book.Page);
    Console.WriteLine("Name: {0} - age: {1}", author.Name, author.Age);
    Console.ReadLine();
}
```

able. [D:\Projects\source\repos4-hust\oop02\oop02.csproj]
Book: Book1 - page: 100
Name: Author1 - age: 50
press any key to exit

Generic delegate

- Generics cũng có thể được sử dụng với delegate, cho phép chúng ta định nghĩa một delegate mà hoạt động với một kiểu dữ liệu chưa biết trước.

```
public delegate T MyGenericDelegate<T>(T arg);
```

- VD: Trong delegate, .Net đã cung cấp sẵn các generic delegate

```
public delegate void Action();  
public delegate void Action<in T>(T obj); //và 16 delegate tương tự với  
SL tham số đầu vào từ 1 đến 16  
public delegate TResult Func<out TResult>();  
public delegate TResult Func<in T, out TResult>(T arg); //và 17 delegate  
tương tự  
public delegate bool Predicate<in T>(T obj);
```

Ràng buộc - constraint

- Generic - kiểu tổng quát; nhưng có lúc cần giới hạn kiểu dữ liệu có thể sử dụng
- Đặt ràng buộc cho generic => compiler báo lỗi nếu vi phạm
 - có thể đặt nhiều ràng buộc
- Cú pháp `GenericTypeName<T> where T : constraint1, constraint2`

- VD

```
public class MyClass<T> where T : class
{
    // T phải là một tham chiếu kiểu dữ liệu (class)
}
```

Ràng buộc - constraint

VD

```
public class Employee
{
    public Employee(string name, int id) => (Name, ID) = (name, id);
    public string Name { get; set; }
    public int ID { get; set; }
}
class EmployeeList<T> where T : notnull, Employee, IComparable<T>, new()
{
    // ...
    public void AddDefault()
    {
        T t = new T();
        // ...
    }
}
```

Tóm tắt

- Generic:
 - Viết mã xử lý không cần phải chỉ rõ kiểu dữ liệu cho đến khi mã được sử dụng
 - Vẫn sử dụng được compiler type checking
- Ưu điểm
 - An toàn cho kiểu dữ liệu (type safe)
 - Viết mã gọn, hiệu quả, tái sử dụng, linh hoạt
 - Hiệu năng

Java Generic programming

- Từ Java 5: cũng đã hỗ trợ generic programming - template
 - Ép kiểu (từ object) => mất kiểm soát kiểu dữ liệu
- Java cung cấp generic class, interface, method

```
public class ClassName<T> {  
    // thuộc tính và phương thức sử dụng T để chỉ định kiểu dữ liệu  
}  
  
public interface InterfaceName<T> {  
    // Khai báo các phương thức sử dụng T để chỉ định kiểu dữ liệu  
}  
  
public <T> void methodName(T parameter) {  
    // code here  
}
```

Java Generic programming

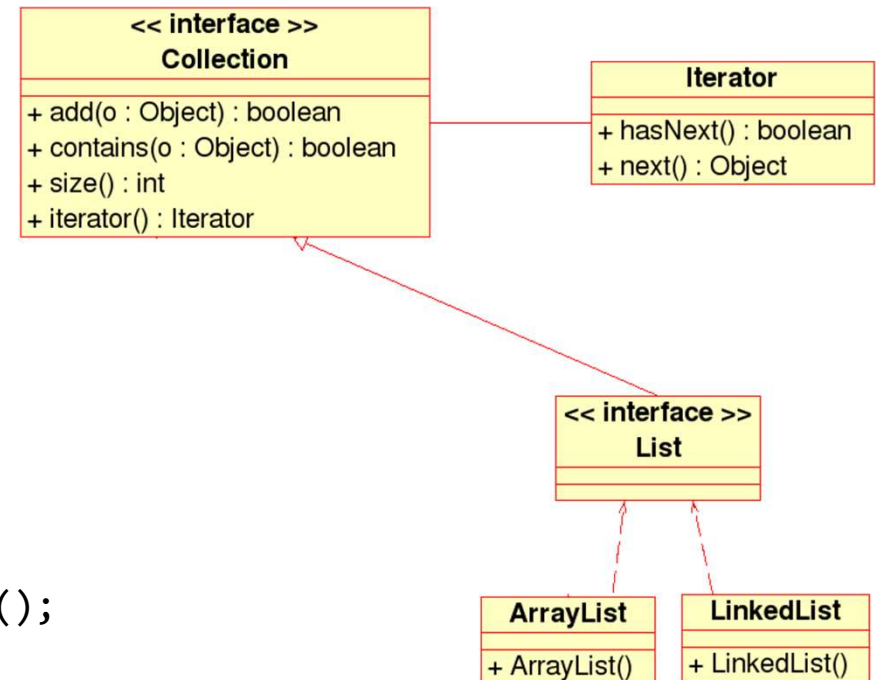
- Cú pháp: dấu <, > ký tự đặc biệt
- Quy ước đặt tên thường dùng:

Tên kiểu	Mục đích
E	Các thành phần trong một collection
K	Kiểu khóa trong Map <key, value>
V	Kiểu giá trị trong Map <key, value>
T	Các kiểu dữ liệu tổng quát
S, U	Các kiểu thông thường khác

Java 1.5 Template

- Không dùng Template

```
List myList = new LinkedList();  
myList.add(new Integer(0));  
Integer x = (Integer) myList.iterator().next();
```

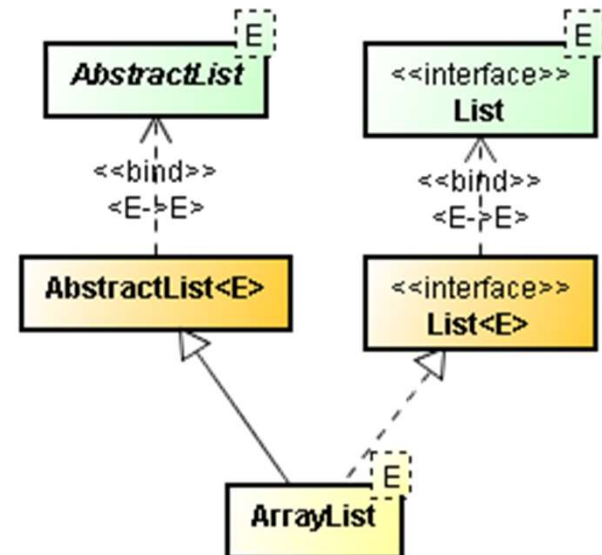


Java 1.5 Template

■ DÙNG Template:

```
List<Integer> myList = new LinkedList<Integer>();  
myList.add(new Integer(0));  
Integer x = myList.iterator().next();  
myList.add(new Long(0)); // Error
```

```
List myList = new LinkedList();  
myList.add(new Integer(0));  
Integer x = (Integer) myList.iterator().next();
```



Generic trong python

- Thường python kh kiểm tra kiểu, biến đưa về kiểu phù hợp giá trị
- v3.5 giới thiệu generic, hỗ trợ type hint
- Hỗ trợ function, class

```
from typing import List, TypeVar, Generic
T = TypeVar('T')
def reverse(items: List[T]) -> List[T]:
    return items[::-1]
```

```
numbers = [1, 2, 3, 4, 5]
reversed_numbers = reverse(numbers)
print(reversed_numbers)
```

```
fruits = ['apple', 'banana', 'cherry']
reversed_fruits = reverse(fruits)
print(reversed_fruits)
```

Bài tập

- Phương thức swap(a,b) đổi chỗ 2 phần tử
 - Viết generic method

```
public static class Swaper{  
    public static void Swap (ref string a, ref string b)  
    {  
        var temp = b;  
        b = a;  
        a = temp;  
    }  
}
```

```
string a = "Hanoi", b = "HaiPhong";  
Console.WriteLine($"Before swap: a = {a}, b = {b}");  
Swaper.Swap(ref a, ref b);  
Console.WriteLine($"After swap: a = {a}, b = {b}");
```